Given two objects represented by the tuples (22, 1, 42, 10) and (20, 0, 36, 8), compute both the Euclidean distance and Manhattan distance between the two objects.

Manhattan Distance: dist = |22-20| + |1 – 0| + |42-36| + |10-8| = 2+1+6+2=11
Euclidean Distance

$$\sqrt{(22-20)^2 + (1-0)^2 + (42-36)^2 + (10-8)^2} = \sqrt{4+1+36+4} = \sqrt{45}$$

Calculate the cosine distance:

```
def cosine(a, b):
    dot_product = sum([a1*b1 for (a1, b1) in zip(a, b)])
    a_square = sum([ai*ai for ai in a])
    b_square = sum([bi*bi for bi in b])
    denominator = math.sqrt(a_square*b_square)
    return dot_product/denominator
a = [1, 2, 3]
b = [4, -5, 6]
print(cosine(a, b))
```

Use z-score normalization to transform the value 35 for age, where the age mean is 29.96 and the standard deviation of age is 12.94 years.

Answer: According to the equation:

$$v' = \frac{v - \mu_A}{\sigma_A}$$

The normalized value will be: (35-29.96)/12.94 = 0.3895

equal-frequency and equal-width partitioning

Apriori algorithm:

1. list all the items (CI_1)
2. filter out infrequent items (FI_1)
3. list all item pairs in FI_1 so we have CI_2
4. filter out infrequent item pairs(FI_2)
5. repeat

find the strong association rule
for item pair in FI_k:
    list all candidate rules
    #e.g. {bread,milk} can have two rules: bread->milk and milk->bread
    Check if it is a valid rule by calculate the support: sc(bread)/sc(bread,milk)

FP-growth algorithm

1. List all the items and discard infrequent ones
2. Sort the list in descending order
3. Build the fp-growth tree

4. Build the fp-growth table

Find the strong association rule:

1. X buys item1 and item2 ->buy item3
2. Pick the frequent pattern includes (item 1,2 and 3) e.g.{k,e,o}
3. List all the candidate rule: k,e->o k,o->e e,o->k
4. Calculate the confidence: sc(k,e,o)/sc(k,e), keep the rules that meet the minimum confidence requirement.

Seasonal index and deseasonalize the data

Given the sequence data, write down all 2-sequences

1. List all items and discard infrequent ones
2. List all combination: a and b can generate {a}{b} and {a,b} two sequences, which is different of apriori: a and b ->{a,b}

List k-subsequence and k-element subsequence
k-subsequence: subsequence with k item
<{1,2},{3,4}>: 4 item
k-element subsequence:subsequence with k element(brace)
<{1,2},{3,4}>: 2 element

Given two document, calculate the TF-IDF score

Use my code snippet given in the solution or check the definition in the slide

Given a table such as

Rank *i*	+/–	*p*(*i*)	*r*(*i*)
1	+	1/1 = 100%	1/8 = 13%
2	+	2/2 = 100%	2/8 = 25%
3	+	3/3 = 100%	3/8 = 38%
4	–	3/4 = 75%	3/8 = 38%
5	+	4/5 = 80%	4/8 = 50%
6	–	4/6 = 67%	4/8 = 50%
7	+	5/7 = 71%	5/8 = 63%
8	–	5/8 = 63%	5/8 = 63%
9	+	6/9 = 67%	6/8 = 75%
10	+	7/10 = 70%	7/8 = 88%
11	–	7/11 = 63%	7/8 = 88%

Calculate the precision and recall

Use my code snippet in the solution (save a great amount of time)
Usage: precision = precision('+-+-++-+--')
recall = recall('+-+-++-+--'

Calculate the pearson similarity and Jaccard similarity

Pearson similarity:
def pearson(a,b):

```
    return cosine_similarity((a-a.mean()),(b-b.mean())
```

Jaccard similarity:

```
def jaccard(a,b):
    return len(a.intersection(b))/len(a.union(b))
```

tips: replace missing value with 0

Use item-item pearson similarity to predict the user's rating on item_target:

Calculate the pearson similarity of each item pair <i_target, i_k>

Pick the top-k item with highest pearson similarity

Return the weighted average of rating:$(w_1*r_1+w_2*r_2)/(w_1+w_2)$